

SAÉ 2.1 – Développement d'un jeu en Java

Projet : Démineur - “The Great Démineur”

Prénom / NOM :

Djibril GUINET

Anthony MILLERIOUX

Groupe : 1

Date de rendu : 12 avril 2026

Sommaire

1. Introduction
2. Présentation générale du projet
3. Fonctionnalités du programme
4. Structure du programme
5. Mécanisme de sauvegarde d'une partie
6. Algorithme de révélation en chaîne
7. Difficultés rencontrées et choix techniques
8. Conclusion personnelle
9. Annexes

1. Introduction

Dans le cadre de cette SAÉ, nous devions réaliser en Java un jeu **Démineur**, en utilisant uniquement nos propres classes ainsi que l'API officielle de Java vue en cours. Le projet devait également être accompagné d'un rapport expliquant son fonctionnement, sa structure et certains choix de conception.

Le principe du démineur est simple, une grille contient un certain nombre de mines placées aléatoirement. Le joueur doit révéler toutes les cases qui ne contiennent pas de mine. Lorsqu'une case sûre est révélée, elle affiche le nombre de mines présentes dans le carré de mine autour d'elle. Si le joueur clique sur une mine, la partie est perdue.

Notre version propose une interface graphique en Java Swing avec plusieurs menus, une configuration de la partie, un système de sauvegarde, ainsi que la logique de révélation automatique des zones vides.

2. Présentation générale du projet

Notre projet sépare clairement :

- la **logique du jeu**, gérée principalement par la classe Demineur ;
- les **données d'une case**, représentées par la classe Case ;
- la **gestion de la sauvegarde**, assurée par SaveManager ;
- l'**interface graphique**, répartie entre plusieurs écrans (MainMenu, ConfigScreen, GameScreen, EndScreen) ;
- la **gestion des interactions utilisateur**, notamment les clics sur les cases, gérée par CaseClickListener.

L'application démarre sur un **menu principal** qui permet de :

- commencer une nouvelle partie ;
- reprendre une partie sauvegardée ;

- quitter le programme.

Lorsqu'une nouvelle partie est lancée, l'utilisateur choisit :

- la largeur de la grille ;
- la hauteur de la grille ;
- le nombre de mines.

La partie se déroule ensuite sur une grille où le joueur peut révéler des cases ou poser des marqueurs pour l'aider à repérer les mines à l'aide du clic gauche et du clic droit.

3. Fonctionnalités du programme

3.1 Menu principal

Au lancement, l'application affiche un menu principal avec trois boutons :

- Nouvelle partie ;
- Reprendre la partie ;
- Quitter .

Le bouton de reprise est fonctionnel que si une sauvegarde existe déjà. Cela évite de proposer une option inutilisable.



3.2 Écran de configuration

L'écran de configuration permet au joueur de définir les paramètres d'une nouvelle partie :

- largeur de la grille ;
- hauteur de la grille;
- nombre de mines.

Des boutons permettent d'augmenter ou diminuer les valeurs par écart de 1 ou de 10. Le programme aussi impose des limites :

- dimensions entre 4 et 30 ;
- nombre de mines inférieur au nombre total de cases.

Cette étape permet de rendre le jeu plus flexible, en autorisant différentes tailles de plateaux.

CONFIGURATION

Largeur : 10

Hauteur : 10

Mines : 30

3.3 Grille de jeu

L'écran principal de jeu affiche une grille composée de cases interactives.

Chaque case peut être:

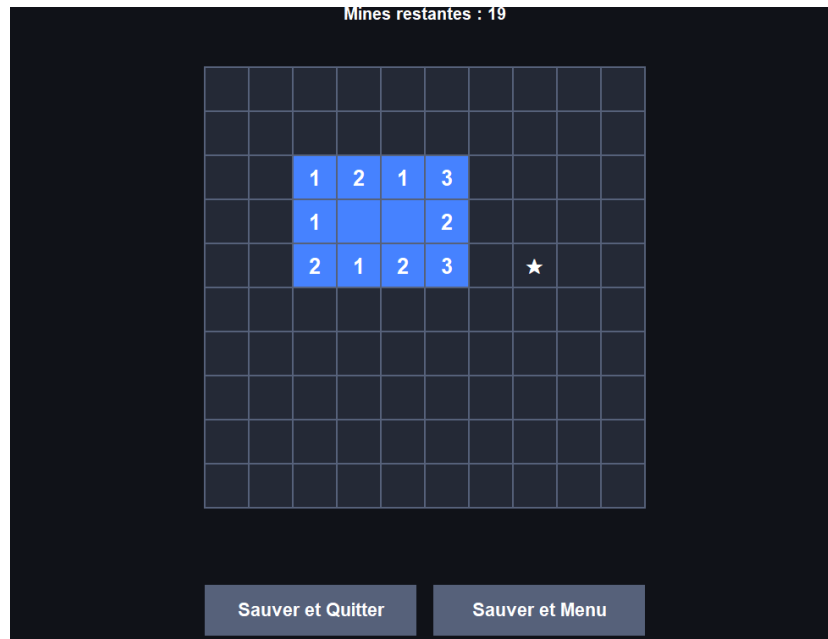
- Cachée ;
- Révélée ;
- Marquée par un symbole d'aide.

Un clic gauche sur une case cachée permet de la révéler. Si la case contient une mine, la partie est perdue. Sinon, elle affiche le nombre de mines présentes autour d'elle. Lorsque ce nombre vaut zéro, la case ne montre aucun chiffre et le programme révèle automatiquement les cases voisines.

Un clic droit permet de changer le marqueur sur une case cachée selon le cycle suivant :

vide → ★ → ? → vide

Ces marqueurs servent seulement d'aide visuel pour le joueur.



3.4 Première case sécurisée

Une fonctionnalité importante de notre projet est la protection au premier clic. Les mines ne sont pas placées dès la création de la grille, mais seulement au moment de la première révélation.

Par conséquent, la première case cliquée ne peut jamais contenir de mine. Cela évite qu'une partie soit perdue immédiatement par malchance, ce qui améliore l'expérience de jeu de l'utilisateur.

3.5 Fin de partie

Le programme gère deux fins de partie :

- **victoire** : toutes les cases non minées ont été révélées ;
- **défaite** : une mine a été cliquée.

Après une victoire, un écran spécial affiche un message de succès, le temps écoulé et la progression réalisée.

Après une défaite, le programme met visuellement en évidence la mine déclenchée, puis révèle les autres mines avant d'afficher un écran de fin.

VICTOIRE !

Temps écoulé : 29 secondes

Score : 98 sur 98 cases

[Retour au menu](#)

DOMMAGE...

Temps écoulé : 5 secondes

Progression : 3 sur 70 cases

Retour au menu

3.6 Sauvegarde et reprise

Pendant une partie, le joueur peut sauvegarder l'état du jeu grâce aux boutons :

- Sauver et Quitter
- Sauver et Menu

La sauvegarde permet de reprendre plus tard une partie laissée en cours. Lorsqu'une partie est gagnée ou perdue, la sauvegarde est supprimée afin d'éviter de reprendre une partie déjà terminée.

4. Structure du programme

Notre programme est organisé en plusieurs classes spécialisées, chacune avec un rôle précis. Cette organisation améliore la lisibilité, limite les responsabilités de chaque classe et facilite la maintenance.

4.1 Vue d'ensemble

Les classes principales sont les suivantes :

Display

Cette classe joue le rôle de point d'entrée de l'application. Elle crée la fenêtre principale et gère la navigation entre les différents écrans.

MainMenu

Elle construit l'interface du menu principal.

ConfigScreen

Elle gère l'écran de configuration d'une nouvelle partie.

GameScreen

Elle construit l'écran de jeu, c'est-à-dire la grille et les boutons de sauvegarde.

EndScreen

Elle affiche les écrans de victoire et de défaite.

NiceButton

Cette classe permet de contrôler l'apparence de tous les boutons personnalisés utilisés dans l'interface.

Case

Elle représente une case de la grille. Elle contient ses coordonnées, son état, la présence éventuelle d'une mine, le nombre de mines voisines et le marqueur posé par le joueur.

Demineur

Il s'agit de la classe centrale de la logique du jeu. Elle gère :

- la grille ;
- le placement des mines ;
- la révélation des cases ;
- les conditions de victoire et de défaite ;
- le temps de jeu ;
- la sauvegarde.

Marqueur

Cette classe regroupe les constantes et les méthodes utilitaires liées aux marqueurs.

SaveManager

Elle prend en charge l'enregistrement et la reconstruction d'une partie sauvegardée.

CaseClickListener

Cette classe relie l'interface graphique à la logique du jeu. Elle interprète les clics de souris sur les cases et fait ensuite les actions nécessaires.

4.2 Organisation logique

Il y a 3 grand ensemble de classes :

a) Logique de jeu

- Demineur
- Case
- Marqueur
- SaveManager

b) Interface graphique

- Display
- MainMenu
- ConfigScreen
- GameScreen
- EndScreen
- NiceButton

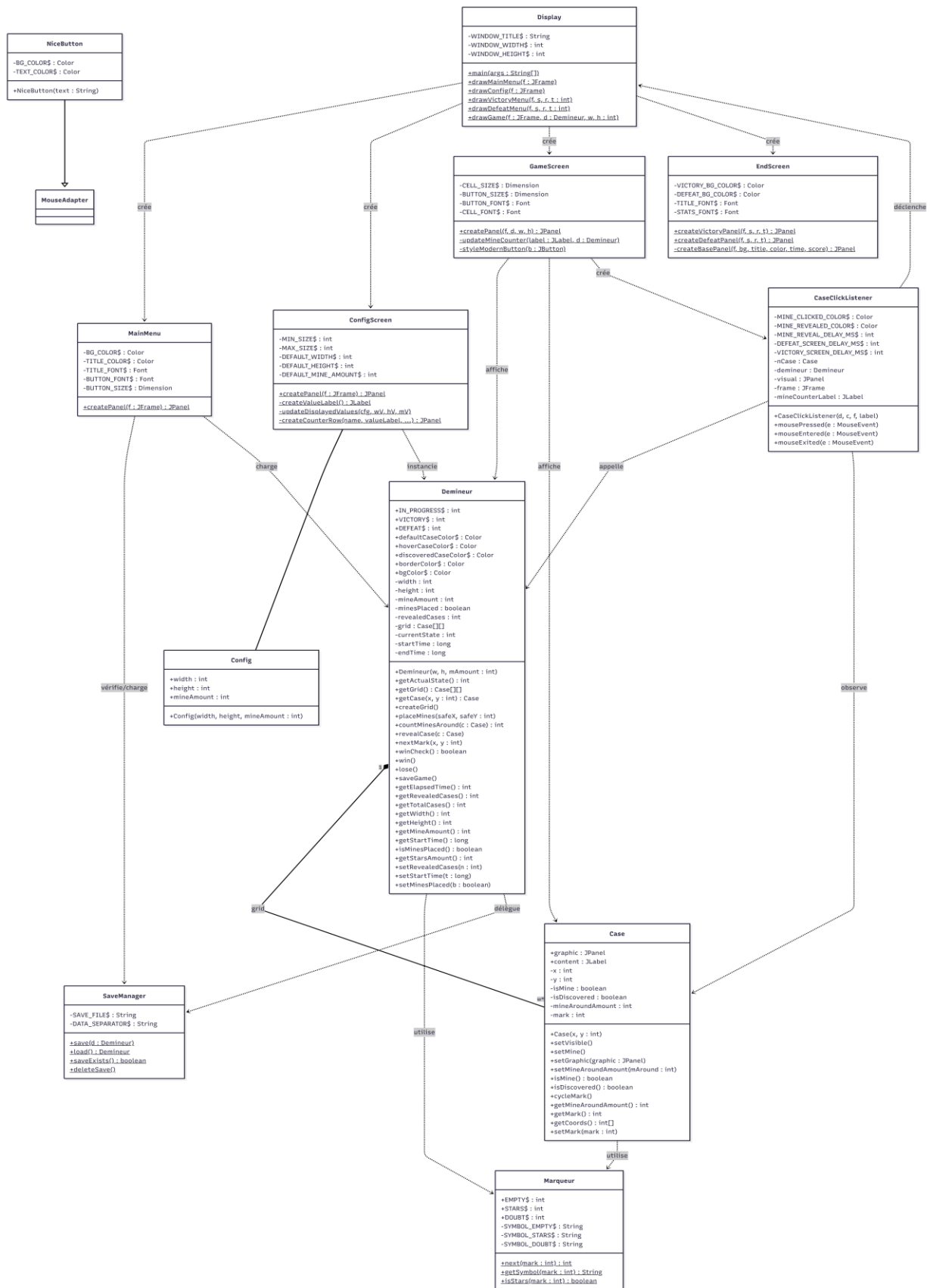
c) Contrôle des événements

- CaseClickListener

Notre séparation s'approche grandement d'une architecture simple de type MVC :

- **modèle** pour les données et la logique ;
- **vue** pour l'affichage ;
- **contrôleur** pour les interactions.

4.3 Diagramme de classes



Le diagramme de classes met en évidence le rôle central de Demineur, qui manipule un tableau de Case et interagit avec SaveManager pour la sauvegarde. Les écrans graphiques sont coordonnés par Display, tandis que CaseClickListener fait le lien entre les actions de l'utilisateur et les et le résultat en jeu.

5. Mécanisme de sauvegarde d'une partie

La sauvegarde repose sur un fichier texte nommé **save.txt**. L'objectif est de mémoriser suffisamment d'informations pour reconstruire une partie exactement dans l'état où elle se trouvait.

Les informations enregistrées sont :

- la largeur de la grille ;
- la hauteur de la grille ;
- le nombre total de mines ;
- le nombre de cases déjà révélées ;
- l'instant de départ de la partie ;
- l'état détaillé de chaque case.

Pour chaque case, la sauvegarde conserve :

- sa position ;
- Si elle est une mine ou non ;
- son état révélé ou non ;
- le nombre de mines adjacentes ;
- le marqueur éventuellement posé.

Lors du chargement, le programme lit ces données puis recrée une nouvelle instance du jeu avec les mêmes paramètres. Ensuite, chaque case de la grille est reconfigurée afin de retrouver l'état exact de la partie sauvegardée.

Ce choix de conception présente plusieurs avantages :

- il est simple à comprendre ;
- il ne nécessite pas de bibliothèque externe ;
- il permet une reconstruction fidèle du plateau.

De plus, la sauvegarde n'est conservée que pour une partie en cours. Si la partie se termine par une victoire ou une défaite, le fichier de sauvegarde est supprimé automatiquement.

6. Algorithme de révélation en chaîne

L'une des mécaniques les plus importantes du démineur est la révélation automatique de plusieurs cases lorsque le joueur découvre une case qui n'a aucune mine autour d'elle.

Dans notre projet, cette fonctionnalité est réalisée par une **approche récursive**.

6.1 Principe général

Lorsqu'une case est révélée :

1. on vérifie d'abord qu'elle existe et qu'elle n'a pas déjà été découverte ;
2. si c'est la première case découverte, les mines sont placées à ce moment-là ;
3. si la case contient une mine, la partie est perdue ;
4. sinon, la case est révélée ;
5. si elle n'a aucune mine adjacente, le programme tente de révéler toutes les cases voisines.

Chaque case voisine applique exactement la même logique. Cela provoque une propagation naturelle dans toute la zone vide, jusqu'à atteindre des cases adjacentes à au moins une mine.

6.2 Pourquoi la récursivité fonctionne ici

La récursivité est bien adaptée à ce problème, car chaque case vide provoque le même traitement sur ses cases voisines. Le programme explore donc progressivement la zone concernée sans avoir besoin d'écrire plusieurs profondeurs de boucles complexes.

Pour éviter les répétitions infinies, la méthode s'arrête immédiatement lorsqu'une case :

- est nulle ;
- a déjà été révélée.

Cette condition est nécessaire, car elle empêche le programme de repasser plusieurs fois par la même case.

6.3 Intérêt pour le joueur

Ce mécanisme rend le jeu plus fluide et plus proche du comportement attendu d'un démineur. Le joueur n'a pas besoin de cliquer manuellement sur toutes les cases d'une grande zone qu'il sait déjà vide, car le programme les révèle automatiquement.

7. Difficultés rencontrées et choix techniques

Plusieurs aspects du projet ont nécessité une réflexion particulière.

7.1 Séparer la logique du jeu et l'affichage

Une difficulté importante consistait à ne pas mélanger la logique du démineur avec l'interface graphique. Les séparer pour ensuite les combiner dans une autre classe, est la méthode que nous avons adopté

après réflexion. Cependant il était complexe de réussir à bien séparer les logiques, et certaines classes savent ce qu'elle ne devrait pas savoir, ce qui est le point négatif que nous aurions aimé corriger si nous avions plus de temps, pour nous améliorer personnellement, et la structure du projet. Nous avons donc réparti les responsabilités au mieux entre des classes de logique (Demineur, Case, SaveManager) et des classes d'affichage (GameScreen, MainMenu, etc) pour les réunir avec le CaseClickListener.

7.2 Gérer correctement le premier clic

Le placement des mines au premier clic est important pour notre jeu. Il fait en sorte que la première action du joueur soit toujours sûre. Pour cela les mines ne sont pas générées lors de la création de la grille, mais seulement lors de la première révélation, ce qui nous a demandé un effort cognitif.

7.3 Reconstituer fidèlement une partie sauvegardée

La sauvegarde devait permettre de reprendre la partie dans le même état. Il fallait donc enregistrer non seulement la grille, mais aussi l'état de chaque case ainsi le temps déjà écoulé et les marqueurs posés. L'enregistrement et la lecture dans un fichier n'était pas complexe en soit, mais un défi nouveau car nous n'avions pas souvent fait cela.

7.4 Construire une interface claire

Nous avons cherché à faire une interface simple à comprendre, avec plusieurs écrans distincts et des boutons assez Nice. L'utilisation d'une classe NiceButton pour créer les boutons nous a permis de tous les créer avec notre propre style directement intégré sur tous.

8. Conclusion personnelle

Conclusion de Djibril GUINET

Ce projet a été très enrichissant pour moi. J'ai pu développer mes compétences en gestion de projet, notamment en organisant la séparation des fichiers et des fonctionnalités. J'ai également amélioré mon niveau en Java d'une manière très ludique avec ce jeu intéressant, en profitant d'une grande liberté d'implémentation tout en respectant les consignes demandées qui nous demandait de réfléchir sans être pour autant ennuyeuses vu la qualité des choses que nous avons apprises en les mettant en place.

J'ai aussi découvert que rédiger des commentaires, documenter et refactoriser du code n'était pas aussi ennuyeux que je le pensais. J'ai réalisé la différence, et j'apprécie vraiment de travailler sur un code propre, structuré et commenté, où les valeurs hard-codées sont remplacés par des variables statiques identifiables facilement.

Finalement, au-delà de l'aspect technique que j'ai renforcé (comme la programmation événementielle et orientée objet), ce projet m'a surtout permis de forger de solides bases en organisation et en communication. Ce sont ces compétences qui me serviront tout au long de ma carrière en tant qu'aspirant chef de projet.

Conclusion de Anthony MILLERIOUX

J'ai beaucoup apprécié réaliser ce projet de développement en Java, qui s'est révélé à la fois intéressant et formateur. Dès le début, la phase de réflexion en groupe a été particulièrement enrichissante, notamment lorsque nous avons dû imaginer l'organisation du projet et la manière de concevoir notre Démineur. Le fait d'élaborer rapidement un diagramme de classes nous a permis de structurer clairement nos idées et d'avoir une vision globale du fonctionnement du programme avant même de commencer à coder.

La programmation en Java m'a également beaucoup plu, en grande partie parce qu'elle repose sur les principes de la programmation orientée objet, que j'apprécie particulièrement. Travailler sur ce projet m'a donc permis de renforcer mes compétences dans ce domaine, tout en approfondissant ma compréhension du langage Java.

Enfin, ce projet m'a permis de mieux comprendre l'importance de la réflexion en amont, de l'organisation et du travail en équipe dans le développement d'une application. Il m'a conforté dans mon intérêt pour la programmation orientée objet et m'a donné envie d'aller plus loin dans l'apprentissage et la réalisation de projets similaires.

Merci pour votre lecture et votre attention. *Anthony & Djibril.*